

TD13 - Induction

Exercice 1. Arbres quelconques

- 1 - Donner un ensemble de règles de construction qui permettent de construire l'ensemble des arbres dont les nœuds internes sont étiquetés par des entiers et ayant 1, 2 ou 3 enfants.
- 2 - Donner une définition de type en OCaml correspondant aux règles de constructions précédentes.
- 3 - Donner un ensemble de règles de construction qui permettent de construire l'ensemble d'arbres d'arité quelconque dont les nœuds internes sont étiquetés par des entiers.
- 4 - Donner une définition de type en OCaml pour les arbres d'arité quelconque.

Exercice 2. Sur l'ordre lexicographique

On considère la fonction OCaml suivante :

```
1 let rec f a b = match a, b with
2   | 0, _ -> 42
3   | 1, p when p >= 7 -> f 1 (p-7)
4   | _ -> f (a-1) (f (a-1) (b+7))
```

- 1 - Montrer la terminaison de la fonction f .
- 2 - Conjecturer le résultat de chaque appel de f (pour tout couple $(n, m) \in \mathbb{N}^2$).
- 3 - Démontrer la conjecture précédente.

Exercice 3. Sur les arbres binaires

On considère la fonction suivante en OCaml :

```
1 type arbre = Vide | Noeud of arbre * arbre
2
3 let rec f a1 a2 = match a1, a2 with
4   | Vide, _ -> a2
5   | Noeud(g1, d1), _ -> f g1 (Noeud(a2, d1))
```

- 1 - Soit l'arbre suivant :

```
1 let a = Noeud(Noeud(Noeud(Vide, Vide), Noeud(Noeud(Vide, Vide),
   Vide)), Noeud(Vide, Vide))
```

Dessiner l'arbre correspondant, et calculer $f\ a\ Vide$.

- 2 - Montrer la terminaison de la fonction f .
 - 3 - Montrer que pour tout arbre a , $f\ (f\ a\ Vide)\ Vide = a$.
- Indication* : Commencer par montrer par induction la propriété suivante :

$$\text{Pour tout arbre } a, b, \quad f\ (f\ a\ b)\ Vide = f\ b\ a$$

TD13 – Induction - Compléments

Exercice 1. [Propriétés des ordres lexicographiques]

On rappelle la définition de l'ordre lexicographique sur l'ensemble E^n des n -uplets d'un ensemble E muni d'un ordre :

Définition

Soit un ensemble E muni d'une relation d'ordre $\leq$ et un entier $n > 0$. L'ordre lexicographique sur les n -uplets de E est défini par

$$((x_i)_{1 \leq i \leq n} \leq (y_i)_{1 \leq i \leq n}) \Leftrightarrow ((x_i)_{1 \leq i \leq n} = (y_i)_{1 \leq i \leq n}) \text{ ou } (\exists N \in [1, n] \mid \forall i < N, x_i = y_i \text{ et } x_N < y_N)$$

On demande ici de prouver les deux résultats de cours suivants.

- 1) Si E est non vide et si l'ordre $\leq$ sur E est un ordre total, montrer que, pour tout $n \in \mathbb{N}^*$, l'ordre lexicographique sur E^n est total.
- 2) Si E est non vide et si l'ordre $\leq$ sur E est bien fondé, montrer que, pour tout $n \in \mathbb{N}^*$, l'ordre lexicographique sur E^n est bien fondé

Exercice 2. [Preuves de terminaison]

On reprend la fonction d'Ackermann, citée dans le cours, et définie en OCaml, par :

```
let rec ack n p = match n, p with
  | 0, p -> p + 1
  | n, 0 -> ack (n - 1) 1
  | n, p -> ack (n - 1) (ack n (p - 1))
```

Démontrer que cette fonction termine pour tout $(n, p) \in \mathbb{N}^2$.

Exercice 3. [Preuves de correction]

On considère la fonction définie ci-dessous.

```
let rec somme a b =
  if a = 0 then b else somme (a-1) (b+1)
```

Démontrer que `somme a b` renvoie la somme des entiers a et b pour tout $(a, b) \in \mathbb{N}^2$.